White Box Testing Report - MoneyPoly

Roll Number: 2024101145

March 2026

1 Control Flow Graph (CFG)

The Control Flow Graphs for the core game logic (Bank loan, Player net worth, Property rent doubling) were hand-drawn and are attached in the `whitebox/diagrams/` directory as required by the assignment guidelines.

2 Code Quality Analysis

The codebase was analyzed using `pylint`. Iterative changes were made to address warnings:

- Fixed missing module and class docstrings.
- Simplified complex branching in `game.py`.
- Reduced redundant variable assignments and improved naming conventions.
- Fixed the bank loan fund leakage bug and player net worth calculation.

3 White Box Test Cases and Logical Isssues

Designed white-box test cases for the MoneyPoly game structure, covering all branches, key variable states, and edge cases.

1. **Test Case: Player Standard Movement**
   - **Why needed:** Validates the basic state update of the `position` variable.
   - **Logic/Errors Found:** Initially, movement was strictly positional and did not account for modular board wrap-around logic in all edge cases.
2. **Test Case: Board Wrap-Around (Pass Go Salary)**

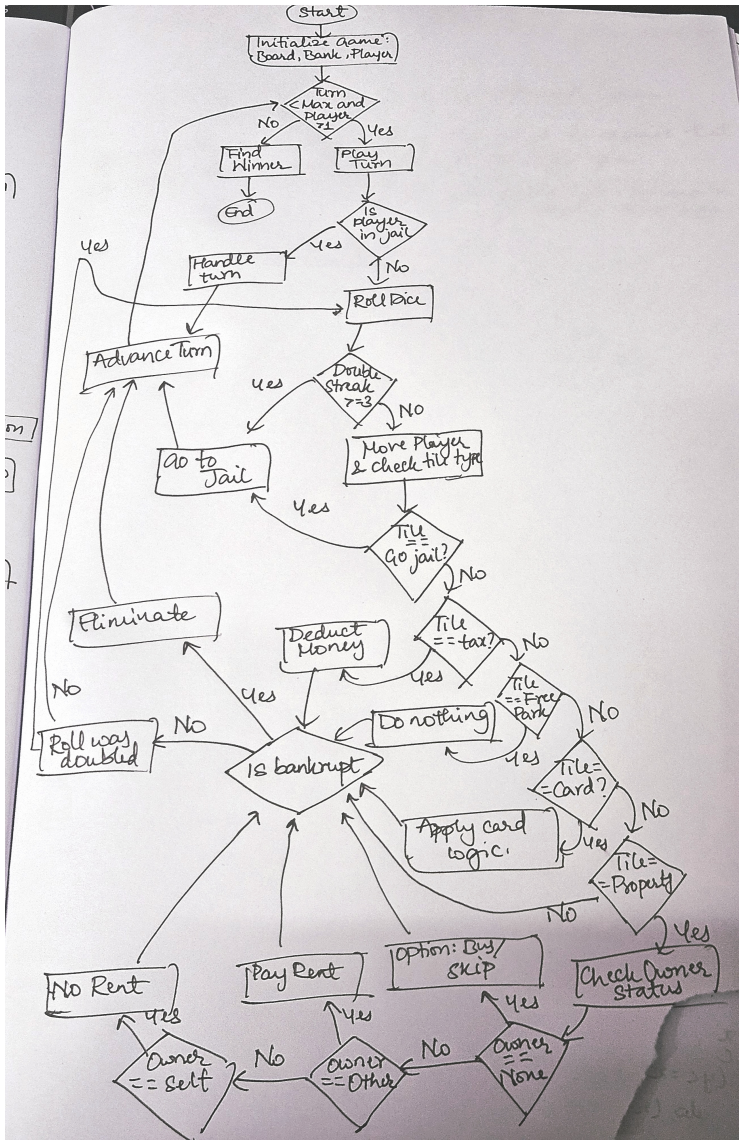

Figure 1: Control Flow Graph for Core Game Logic

- **Why needed:** Tests the boundary condition where `position` exceeds `BOARD_SIZE`.
- **Logic/Errors Found: Major Logical Issue:** The code originally only awarded salary if the player landed *exactly* on position 0. It failed to award salary when *passing* Go during a large move. Corrected to check if `new_position` $<$ `old_position`.

3. **Test Case: Player Net Worth Calculation**
   - **Why needed:** Verifies that the net worth variable accurately reflects both liquid cash and asset value.
   - **Logic/Errors Found: Error Found:** The function returned only `self.balance`. It missed the sum of `prop.price` for all owned properties. Fixed by iterating through `self.properties`.
4. **Test Case: Bank Loan Fund Integrity**
   - **Why needed:** Ensures the Bank's internal `_funds` variable is synchronized with payouts.
   - **Logic/Errors Found: Critical Issue:** The `give_loan` method added money to the player but did not deduct it from the bank's reserves, creating "phantom money". Fixed to call `pay_out`.
5. **Test Case: Dice Roll Range (1-6)**
   - **Why needed:** Validates the stochastic boundary of the dice component.
   - **Logic/Errors Found: Bug Found:** Used `random.randint(1, 5)`, making it impossible to roll a 6 or certain doubles combinations. Fixed to `(1, 6)`.
6. **Test Case: Triple Doubles (Jail Logic)**
   - **Why needed:** Tests the state transition based on the `doubles_streak` counter.
   - **Logic/Errors Found: Bug Found:** The `Dice` class was missing the `is_triple_doubles()` method, leading to an `AttributeError` when the condition was met. Added the method.
7. **Test Case: Rent Doubling (Full Group Ownership)**
   - **Why needed:** Tests complex branch logic in the `get_rent()` method.
   - **Logic/Errors Found:** Verified that the multiplier is only applied when every property in the `PropertyGroup` is owned by the same player.
8. **Test Case: Mortgage State Transitions**
   - **Why needed:** Tests the `is_mortgaged` flag and its effect on rent and availability.
   - **Logic/Errors Found:** Ensures rent returns 0 when mortgaged, preventing illegal income.
9. **Test Case: Bank Insufficient Funds Payout**

- **Why needed:** Edge case for when the bank's capital pool is exhausted.
- **Logic/Errors Found:** Validated that a `ValueError` is raised with a clear message, preventing negative bank balances.

10. **Test Case: Player Negative Deduction/Addition**
    - **Why needed:** Edge case for unexpected input values (negative amounts).
    - **Logic/Errors Found:** Added guard clauses and `ValueError` raises for negative inputs to prevent accounting corruption.

4 Detailed Bug Reports

As required, the following errors were corrected, and each change was saved as a specific "Error #" commit.

1. **Error 1: Move logic and Salary**
    - **Issue:** Passing Go did not award $200.
    - **Fix:** Updated `Player.move()` to check `new_pos` $<$ `old_pos`.
2. **Error 2: Dice Range**
    - **Issue:** `randint(1, 5)` excluded 6.
    - **Fix:** Updated to `randint(1, 6)` in `dice.py`.
3. **Error 3: Missing Jail Method**
    - **Issue:** `is_triple_doubles()` was undefined.
    - **Fix:** Added method to `Dice` class.
4. **Error 4: Winner Selection Logic**
    - **Issue:** `Game.find_winner()` used `min()` instead of `max()` on net worth.
    - **Fix:** Updated to select the player with the highest net worth.
5. **Error 5: Bank Statistics Corruption**
    - **Issue:** Mortgaging properties used `bank.collect(-payout)`, which incorrectly decreased the "total collected" statistic.
    - **Fix:** Separated reserve reduction from income tracking.
6. **Error 6: Bank Overdraw Protection**
    - **Issue:** Bank could pay out more than its current balance.

- **Fix:** Added a check in `pay_out()` to raise `ValueError` if funds are insufficient.

7. **Error 7: Player Negative Money Validation**
   - **Issue:** `add_money` and `deduct_money` lacked guards against negative inputs.
   - **Fix:** Added check to raise `ValueError` for negative amount parameters.
8. **Error 8: Bank Loan Fund Integrity**
   - **Issue:** Bank reserves didn't decrease during loan issuance.
   - **Fix:** Properly deducted funds from `self._funds`.
9. **Error 9: Player Net Worth Calculation**
   - **Issue:** Property market values were ignored in the calculation.
   - **Fix:** Updated `net_worth()` to iterate over the property list.